

LAB 4: LoRa V2.0

La presente evaluación será recibida sólo si se ha cumplido con los checkpoints de todas las preguntas en el **LABORATORIO** con el docente del curso.

La entrega debe constar de:

1. **Introducción** (incluir párrafo introductorio):
 1. **Marco Teórico** (definiciones **con referencias**)
 2. **Estado del Arte** (trabajos de investigación, artículos, tesis, páginas **con referencias**).
2. **Metodología** (explicación de componentes e implementos utilizados y diagramas de flujo/bloques).
3. **Desarrollo de la experiencia** (incluye discusión)
4. **Conclusiones** (una por cada experiencia realizada **como mínimo**)
5. **Referencias** (formato IEEE)

Consideraciones:

- Subir únicamente el PDF del informe a Canvas, usar un link para el repositorio, incluir fotos de paso a paso de cada implementación y un vídeo mostrando el funcionamiento de las experiencias. Sin embargo, para simulaciones, considere la placa Arduino UNO disponible en Tinkercad.
- Considere una placa de Arduino MEGA2560 tal cual existe en el laboratorio.
- Solo un miembro del grupo sube el informe.
- No olvidar colocar los nombres de todos los integrantes.

ÍNDICE

1. Introducción a Comunicación LoRa (Comandos AT)	4
1.1. Instrucciones	4
2. Carga de Código de Prueba a RAK3172S	5
2.1. Instrucciones	5
2.2. Se pide	5
3. Implementación de sistema de comunicación P2P con LoRa	6
3.1. Instrucciones:	6
4. Implementación con Arduino y sensor	8
5. Sistema IoT de alerta de fugas de gas	9
5.1. Módulo Detector	9
5.2. Módulo de Alerta	10
5.3. Muestre al docente	10
6. LoRa	11
6.1. Características	11
7. Dispositivos RAK	11
8. Conexión de dispositivos RAK con Conversor TTL	12
9. RAK SERIAL PORT TOOL	13
10. RAK WisToolBox	13
11. Paquete de soporte de la placa RAK3272S RUI3 en Arduino IDE	16
12. Comandos AT (RUI v3)	20
12.1. RUI3 Formato de comandos AT	20
13. Sensor de Gas MQ2	22
14. Código de Ejemplo Arduino + RAK	24
15. Código Ejemplo de encendido o apagado de LED en RAK como Receptor	25

LoRa - I

Consideraciones iniciales:

- **IMPORTANTE: NIVEL DE ALIMENTACIÓN 3.3V**
- **IMPORTANTE: NUNCA ENERGIZAR SIN LA ANTENA.**
- **IMPORTANTE: AL CARGAR CÓDIGOS NUNCA DESCONECTAR EN PROCESO DE CARGA**

INSTRUCCIONES INICIALES

Como parte de la experiencia de laboratorio se cuentan con los dispositivos:



RAK3172 Breakout Board

Los cuales son de la empresa RAKWireless. Estos dispositivos trabajan con un estándar denominado **RUI3 (Rak United Interface)**, el cual emplea comandos AT para realizar la programación de los mismos.

Información complementaria de cada dispositivo se encuentra através del siguiente enlace [RAK3172 Datasheet](#)

Cabe mencionar que estos dispositivos tienen una muy buena documentación en línea, que abarca casi todos los aspectos de su programación y uso.

Checkpoints

- **C1:** Ejecución de comandos AT en RAK3172 mediante UART.
- **C3:** Comunicación P2P con dos dispositivos RAK3272S.
- **C4:** Medición y comunicación de sensor usando com. UART en RAK3272S y Arduino.
- **C5:** Implementación de sistema IoT de detección y alarma de fuga de gases.

1. Introducción a Comunicación LoRa (Comandos AT)

IMPORTANTE: NIVEL DE ALIMENTACIÓN 3.3V Y NUNCA ENERGIZAR SIN LA ANTENA.

Información adicional de Comandos AT: [RUI3 - AT Command Manual](#)

1.1. Instrucciones

- Conectar mediante el conversor serial ambos dispositivos.
 - Nota Importante: Los dispositivos RAK reciben instrucciones de comandos a través de los puertos **UART2 (RX2 y TX2)**.

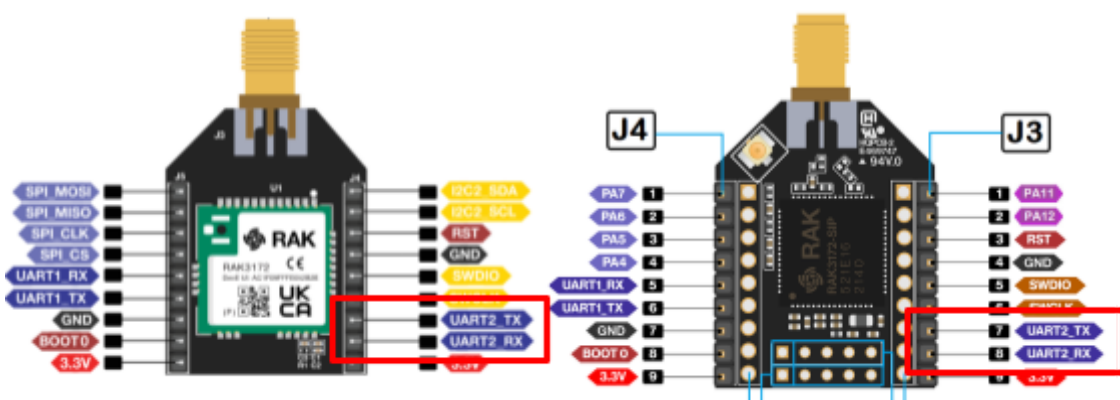


Figura 1. Puertos de comunicación UART.

Nota: Se recomienda usar el RAK Serial Port Tool o el Wistoolbox

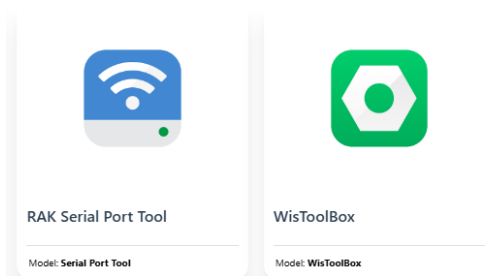


Figura 3. Herramientas de programación directa de dispositivos RAK.

[Enlace a RAK Tools](#)

[WisToolBox](#)

- Realizar la ejecución de los siguientes comandos a través de comunicación serial:
 - AT+VER=?
 - ATZ

CHECKPOINT 01

- Mostrar el resultado de ejecución de código en su dispositivo (adjuntar captura en su informe).
- **Antes de realizar la primera conexión al ordenador, consultar al docente.**

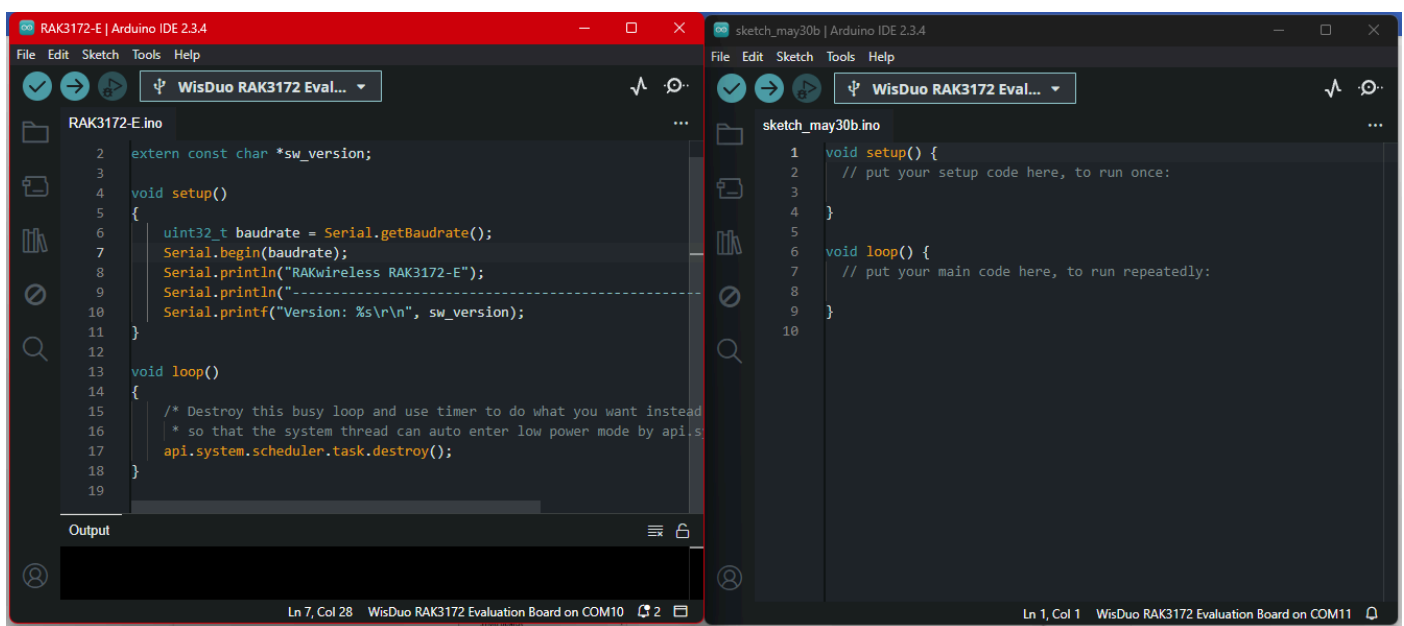
2. Implementación de sistema de comunicación P2P con LoRa

● En esta parte solicitar al docente un segundo RAK y un segundo conversor USB a TTL

- Implemente un sistema de transmisión LoRa P2P utilizando los dos dispositivos RAK:

RAK3272S

- Primero realice la conexión UART con el dispositivo RAK usando el IDE de Arduino.
- Va a tener que programar dispositivo por dispositivo, a fin de poder establecer uno como **TRANSMISOR** y el otro como **RECEPTOR**.
- Tendrá que conectar dos dispositivos a la vez, usando los conversores TTL y tener dos ventanas de Arduino (**se recomienda que sea en dos laptops diferentes**).



(Consultar al docente sobre cualquier duda en implementación).

2.1. Instrucciones:

1. Verificar la correcta conexión enviando el comando AT (o ATZ)
2. Desactivar LoRaWAN y activar modo P2P con el comando:

AT+NWM=0

3. Configurar los parámetros de enlace P2P

AT+P2P=923000000:7:125:0:10:14

- 923000000: frecuencia en Hz

Nota: A fin de aislar las comunicaciones, su grupo usará la banda de frecuencia:

923X00000

Donde X es su número de grupo. Si su grupo es el 3, usted usará la frecuencia 923300000.

- 7: SF7 (Spreading Factor alto = largo alcance)
- 125: ancho de banda (kHz)

- 0: coding rate 4/5
- 10: longitud del preámbulo.
- 14: potencia de transmisión (dBm)

4. RECEPTOR

- Después de configurar el modo P2P, activa la recepción continua:

AT+PRECV=65534

5. TRANSMISOR

- Después de configurar el modo P2P, activa la transmisión:

AT+PRECV=0

6. Envío de mensaje:

AT+PSEND:<Mensaje en Hexadecimal>

```
Output Serial Monitor x
TRANSMISOR
New Line 115200 baud
OK
OK
RAKwireless RAK3172-E
-----
Version: RUI_4.2.1_RAK3172-E
Current Work Mode: LoRa P2P.
OK
OK
OK
+EVT:TXP2P DONE
OK
+EVT:TXP2P DONE
OK
+EVT:TXP2P DONE

Ln 7, Col 28 WisDuo RAK3172 Evaluation Board on COM10

Serial Monitor x
RECEPTOR
New Line 115200 baud
+EVT:RXP2P:-27:13:000000
+EVT:RXP2P:-26:13:000000
+EVT:RXP2P:-26:13:000000

Ln 1, Col 1 WisDuo RAK3172 Evaluation Board on COM11
```

CHECKPOINT 02

3. Implementación con Arduino y sensor

1. Elija uno de los dispositivos para que sea conectado con el Arduino MEGA. La comunicación deberá hacerla a través del puerto **Serial1 del Arduino y el Serial2 del RAK.**
2. Debe de hacerlo en laptops diferentes.
3. Escoger un **sensor cualquiera**, que sea medido con el Arduino MEGA y enviar el mensaje usando LoRa al dispositivo RAK Receptor.

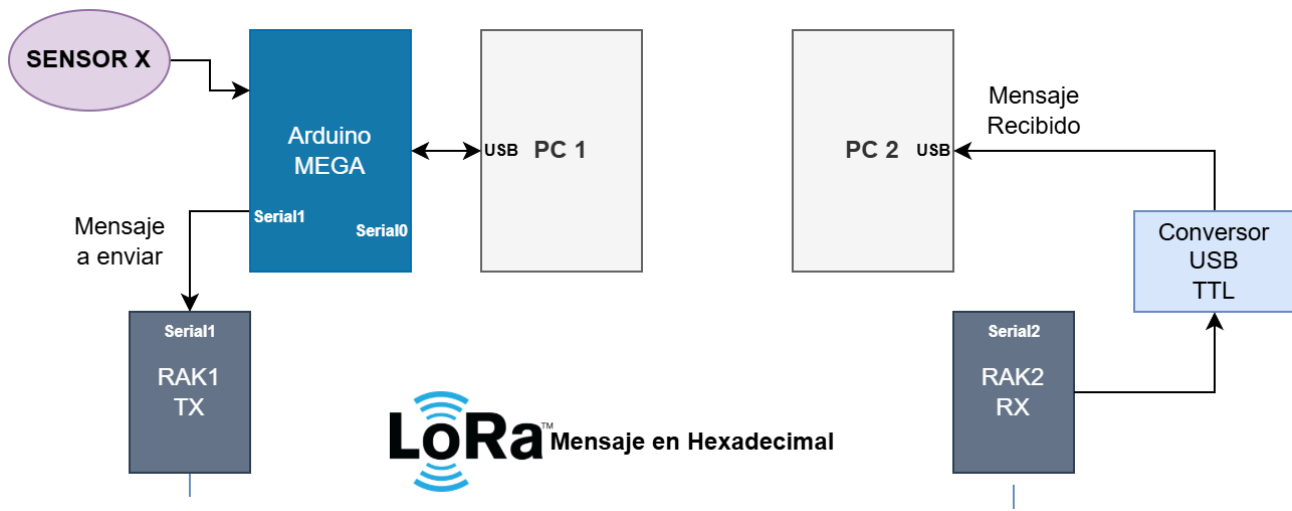


Figura 4. Diagrama de comunicación

4. Implemente un código que haga el envío de un mensaje LoRa (Recuerde que debe estar codificado en hexadecimal). Debe ser visible en el monitor serial conectado al dispositivo receptor.

En la última sección se deja un ejemplo de código.

CHECKPOINT 03

Nota. Recuerde realizar la conversión de datos a Hexadecimal.

USE COMO REFERENCIA LA GUÍA DE CÓDIGO EJEMPLO DE LA SECCIÓN 14 DEL ANEXO.

Adicionalmente se brindan las siguientes guías de conexión y programación del RAK3172S.

[LoRa P2P RAK3172](#)

[AT Commands](#)

[Datasheet 3172S](#)

4. Sistema IoT de alerta de fugas de gas

Debe implementar un sistema IoT que tenga los siguientes componentes (mostrado en la fig. 5):

- Sensor de gas MQ3
- Buzzer
- 2 Módulo LoRa RAK3272S
- Sensor de temperatura LM35
- Micro servo
- Conversor USB-TTL
- Diodos LED

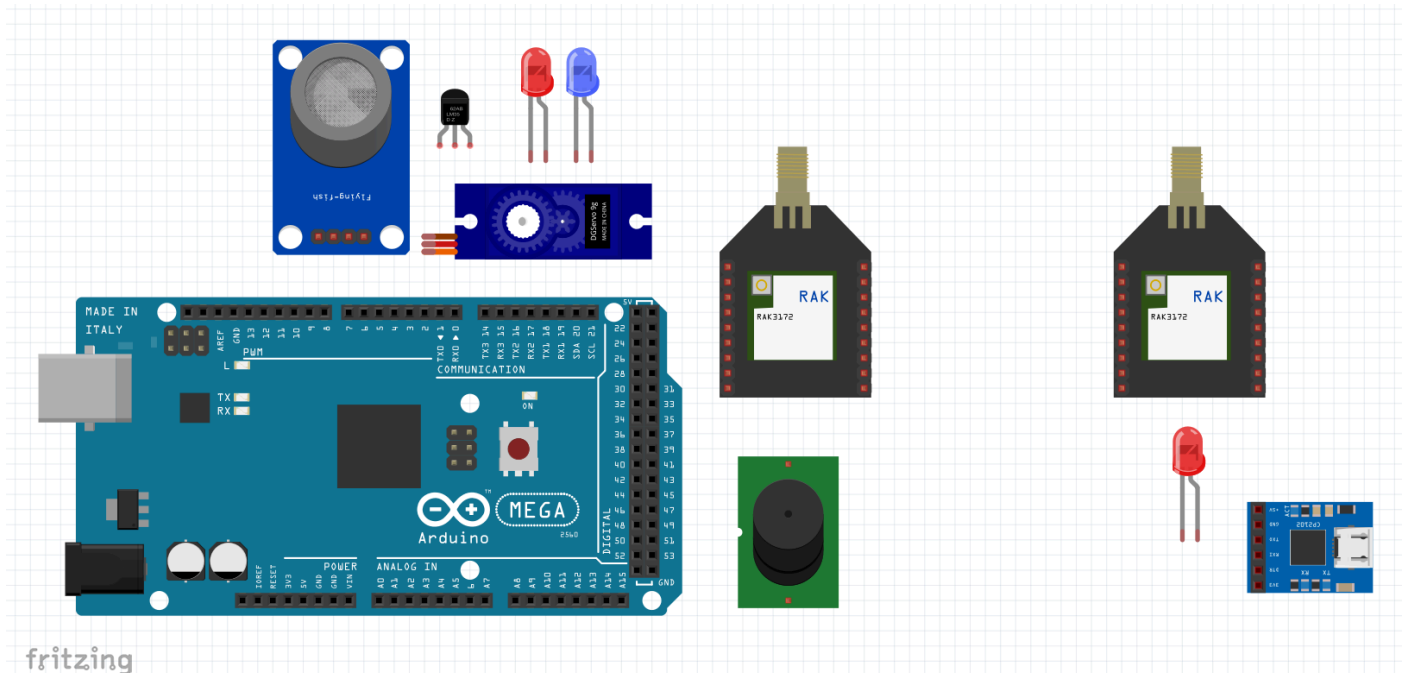


Figura 5. Componentes a emplear en su sistema (izq. módulo detector y der. módulo de alerta).

El sistema a diseñar consta de un sistema de monitoreo y alerta contra posibles fugas de gas dentro de unas instalaciones subterráneas, donde la conectividad WiFi dada la estructura del complejo no es muy eficiente. Este sistema consta de dos partes: el módulo detector y el módulo de alarma.

4.1. Módulo Detector

- Este módulo se encarga de obtener mediciones y evaluar si se está produciendo una fuga de gas.
- Los criterios son:
 - Una temperatura mayor a 50°, que indica un posible fallo en el sistema regulador
 - Presencia de gas: concentración mayor a 80 ppm (calibración para LPG)
- En caso se cumpla alguno de los dos criterios, este módulo deberá:
 - **Alta temperatura:** Encender un LED ROJO de manera intermitente indicando alta temperatura. Así mismo un buzzer deberá accionarse en igual intermitencia que el LED. 500 ms encendido, 1 s apagado.
 - **Fuga de gas:** Encender un LED AZUL de manera intermitente, al igual que el buzzer. En esta configuración el encendido debe ser de 1500 ms, y con 1000 ms de apagado (debe notarse diferencia entre ambos casos al accionar el buzzer).

- En caso de fuga de gas, deberá encender el servomotor que simboliza la activación de una esclusa de aire para ventilación al exterior. Debe pasar de 0° a 180° en este caso.
- Enviar un mensaje a través de LoRa alertando cada tipo de incidencia, según sea el caso.

4.2. Módulo de Alerta

Este se compone del dispositivo RAK usado como receptor. Este debe recibir los mensajes del módulo de alarma (transmisor) y recibirlos en el terminal receptor. Tiene que recibir los mensajes y según el caso mostrar los siguientes mensajes en el terminal.

- Si se ha producido una fuga de GAS, mostrar un mensaje "FUGA DE GAS" y así mismo la lectura del sensor de gas en ppm.
- Si se ha producido una alta temperatura, indicar un mensaje de alerta: "PELIGRO" y la lectura del sensor de temperatura.

Opcional: Puede realizar la recepción y procesamiento de los mensajes en el dispositivo receptor (computadora) usando python. Esto se logra mediante la librería Pyserial. Permite recepcionar los mensajes en un determinado puerto serial (COM) y estableciendo el baudrate.

Recuerde que los mensajes LoRa recepcionados en el RAK tienen la estructura:

+EVT:RXP2P:POT:SNR:PAYLOAD

- POT: La potencia de recepción del mensaje en dBm.
- SNR: La relación de señal a ruido en dBm.
- PAYLOAD: El mensaje en Hexadecimal.

Más información de esta implementación con Pyserial en el anexo respectivo.

CHECKPOINT 01

4.3. Muestre al docente

- **Implementación** de ambos sistemas y **demostrar el funcionamiento** de ambos.
- **Mensajes enviados**, en sus respectivas frecuencias.
- Los sistemas deben estar conectados a **laptops diferentes**, a fin de demostrar la comunicación inalámbrica.
- **Diagrama de bloques y diagrama de flujo** (en clase).
- Tomar **fotos** y grabar **videos** del funcionamiento **para su informe**.

ANEXOS

5. LoRa

Es una tecnología de modulación patentada por Semtech que permite la transmisión de datos de largo alcance (varios kilómetros) utilizando frecuencias de radio no licenciadas. LoRa se centra en la capa física de la comunicación inalámbrica y proporciona una forma eficiente de transmitir datos en entornos con interferencias y con un consumo de energía relativamente bajo.

5.1. Características

- **Largo alcance:** Puede alcanzar distancias de varios kilómetros a decenas de kilómetros, dependiendo de las condiciones y la potencia de la señal.
- **Baja potencia:** Las transmisiones LoRa consumen muy poca energía, lo que las hace ideales para dispositivos alimentados por baterías con una larga vida útil.
- **Penetración de obstáculos:** La capacidad de penetración a través de edificios y otros obstáculos es una ventaja clave en entornos urbanos.
- **Ancho de banda estrecho:** Utiliza un ancho de banda de espectro estrecho, lo que lo hace adecuado para aplicaciones de baja velocidad de datos.

6. Dispositivos RAK

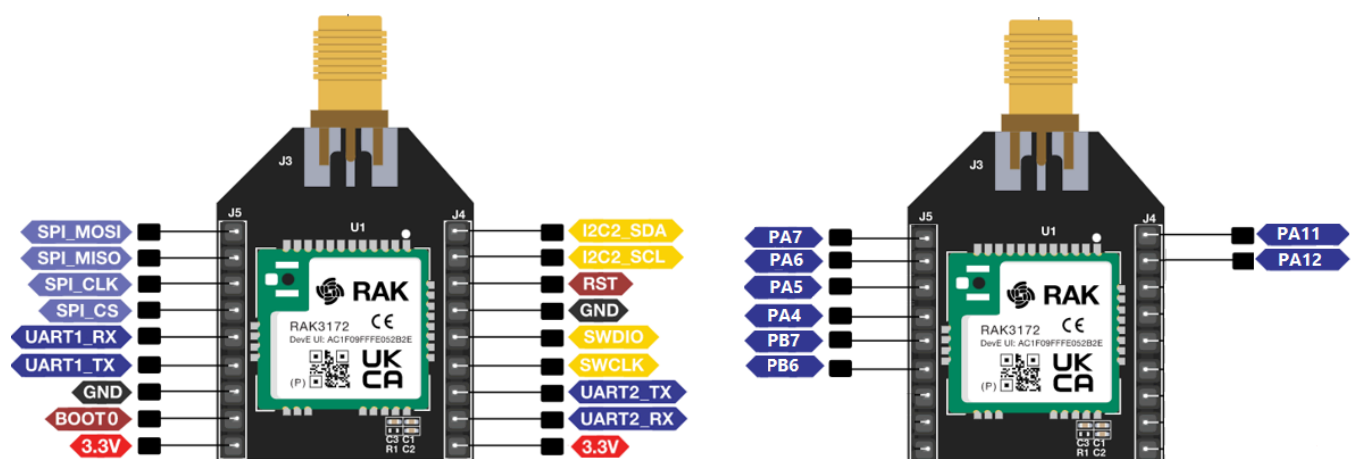


Figura 7. Pinout de dispositivo RAK3172-SiP.

Más información: [RAK3172S](#)

7. Conexión de dispositivos RAK con Conversor TTL

Se debe conectar los dispositivos RAK a través de comunicación UART, usando un conversor USB a TTL.

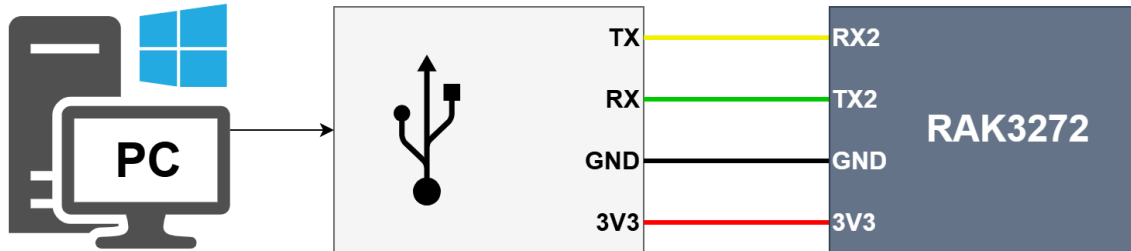
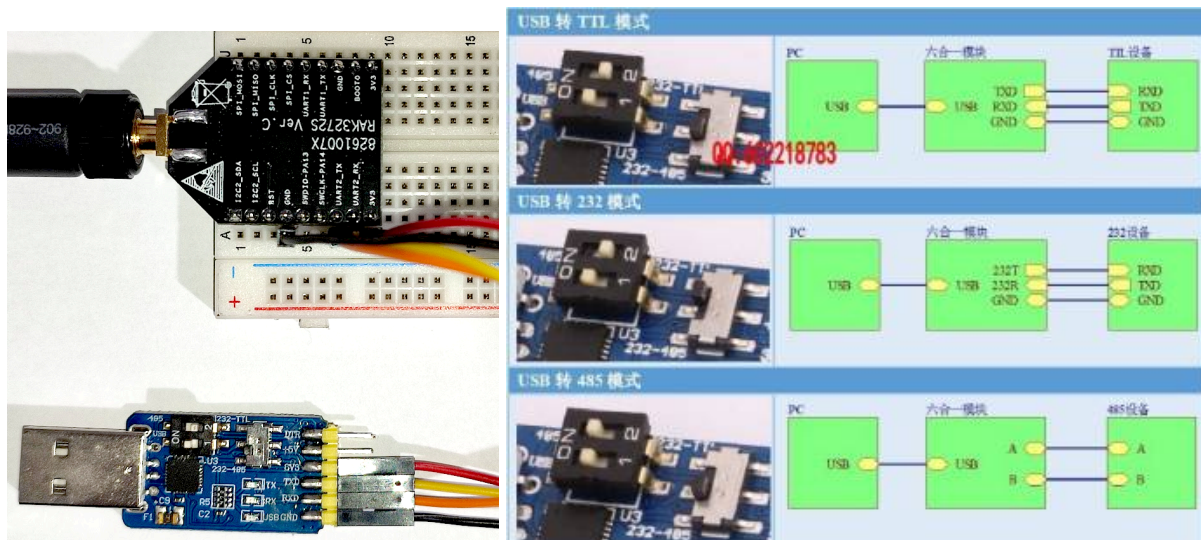


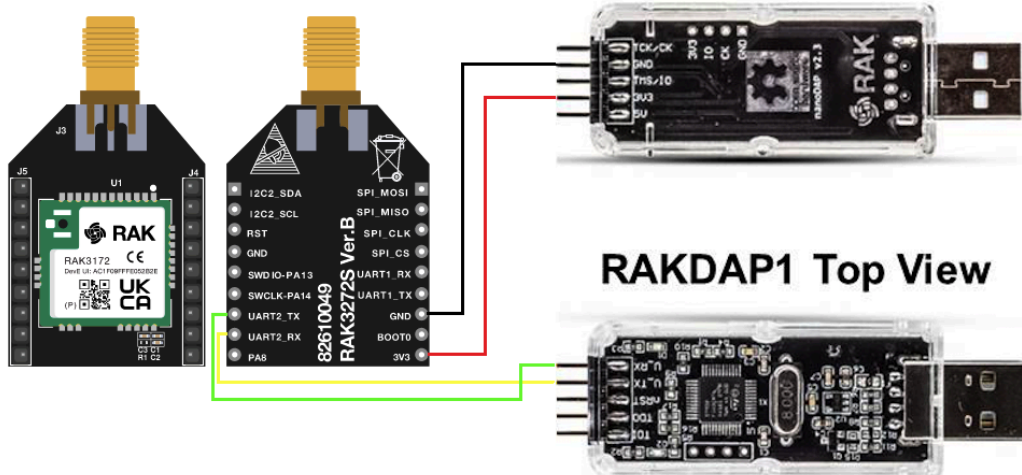
Figura 2. Esquema de conexión de PC a RAK usando conversor TTL.

Tome en consideración que los puertos de comunicación Serial2 (TX2 y RX2) en el dispositivo RAK son los únicos puertos mediante los cuales puede enviar comandos AT (RUIv3).



Ejemplo de conexión con RAK3172S a través de conversor TTL 6 en 1.

RAKDAP1 Bottom View



RAKDAP1 Top View

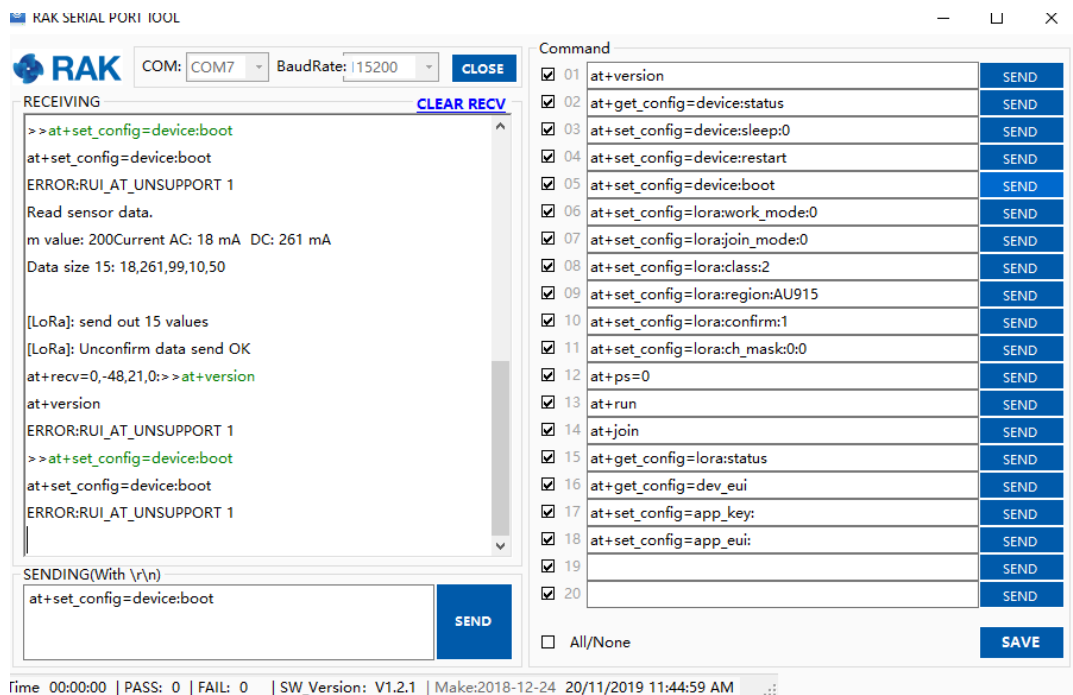
Ejemplo de conexión con RAK3172S a través de RAK DAP.

8. RAK SERIAL PORT TOOL

Nota: Es posible que pida instalar .NET al ejecutar el programa.

.NET Framework 3.5 (includes .NET 2.0 and 3.0)

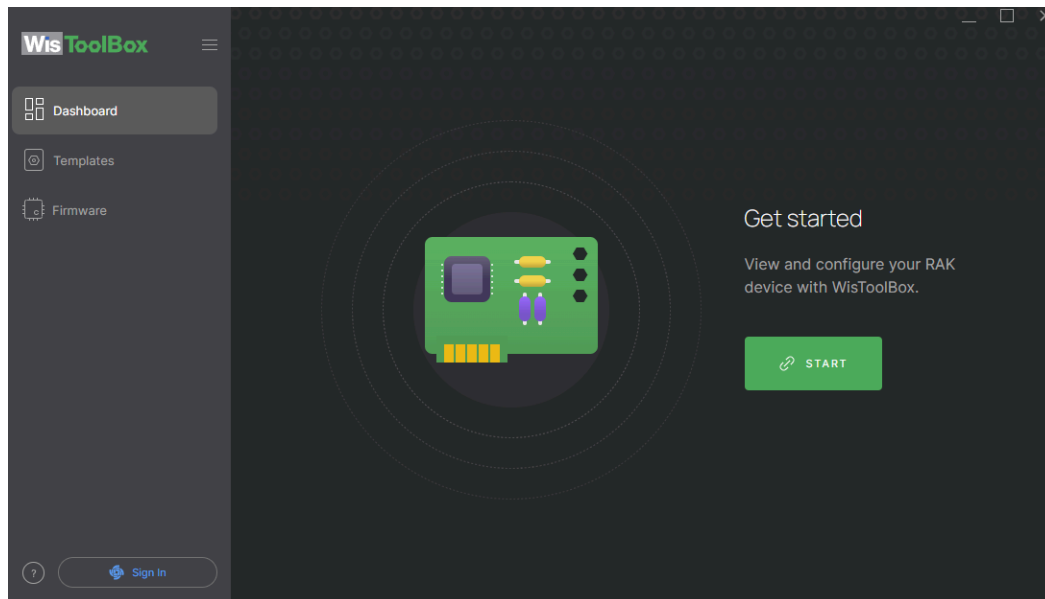
Esta herramienta permite el envío directo de comandos AT a través comunicación UART. Se debe elegir el puerto serial correspondiente (COM) y establecer la tasa de baudios (por defecto: 115200). Luego, se envían los comandos RAK a través de la entrada de texto. Recomendación: enviar comandos siempre en mayúsculas.



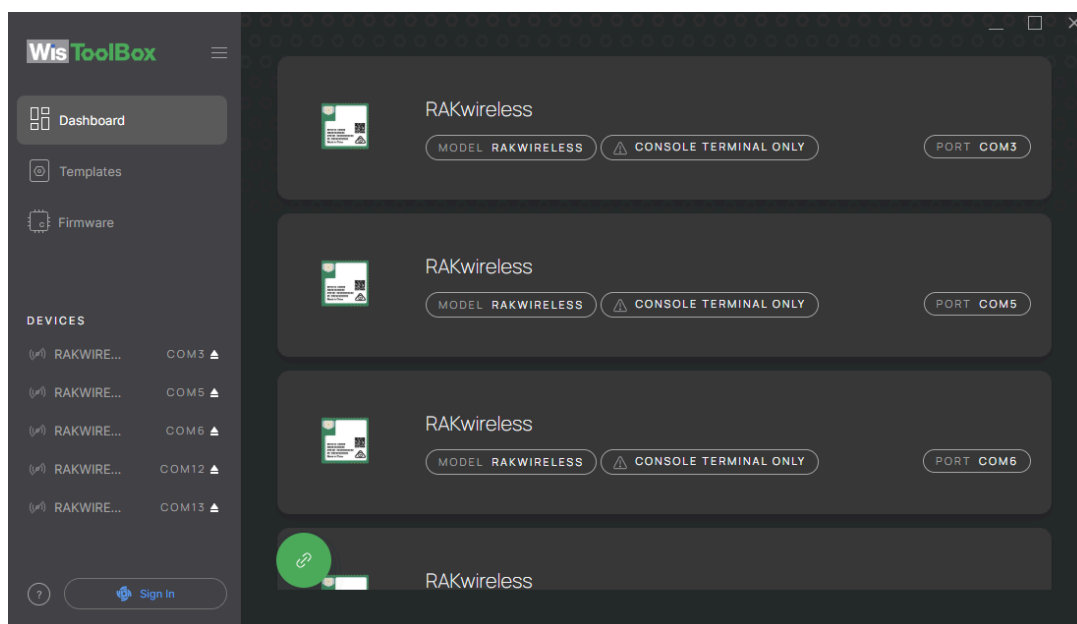
Ventana de RAK Serial Port Tool.

9. RAK WisToolBox

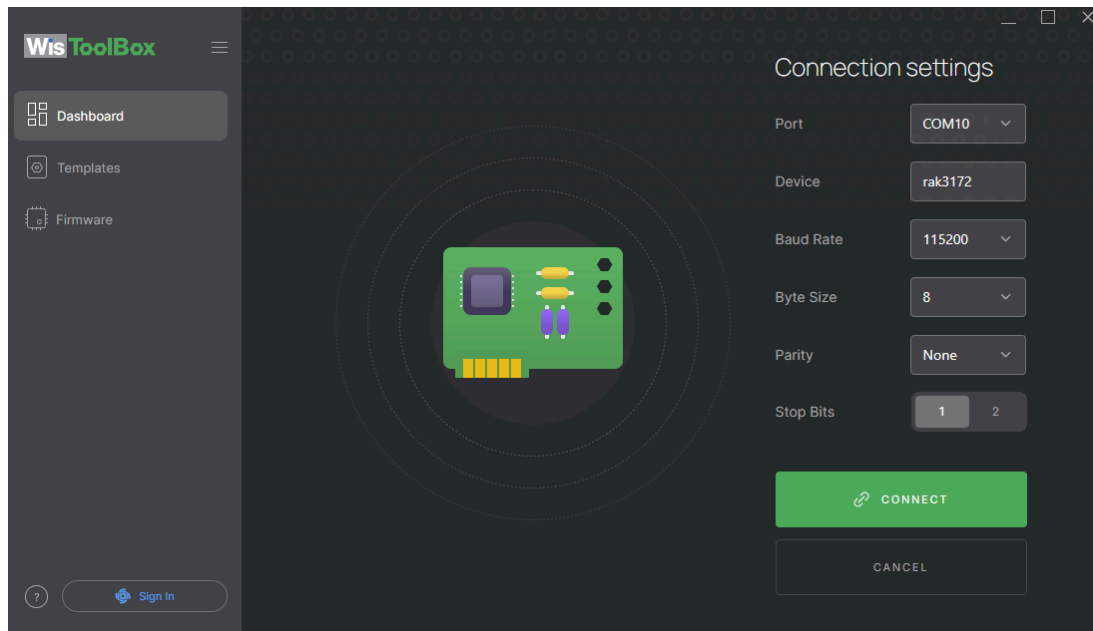
Esta herramienta contiene soporte para dispositivos de RAK más recientes, permitiendo configurar más parámetros de la comunicación serial así como posibilidad de cargar firmware. La programación usando comandos AT se debe realizar mediante la consola de comandos que ofrece este software al conectar un dispositivo RAK. Igualmente se debe establecer tanto el puerto serial como la tasa de baudios; COM y Baudrate respectivamente.



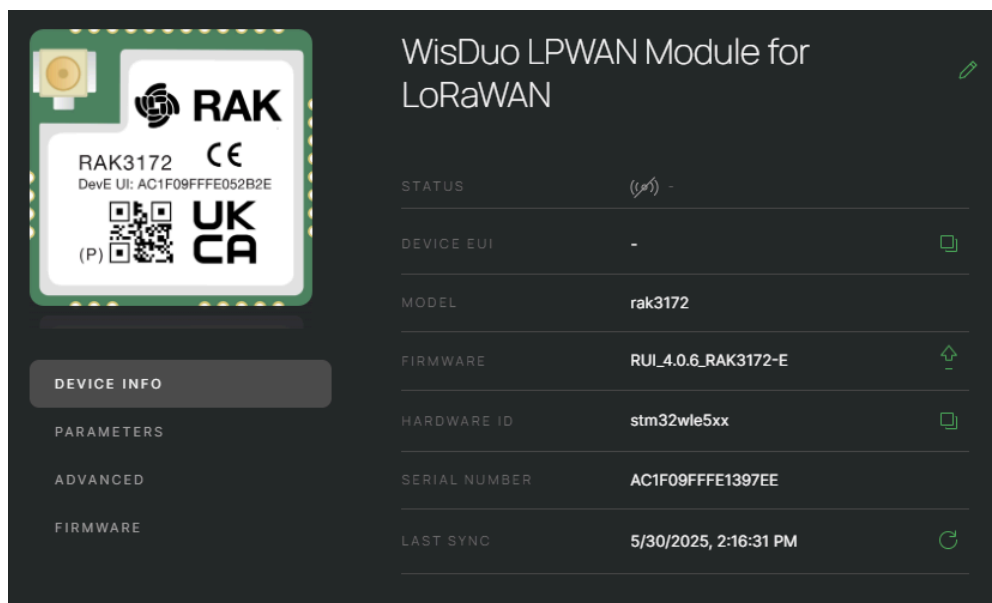
Software WisToolBox



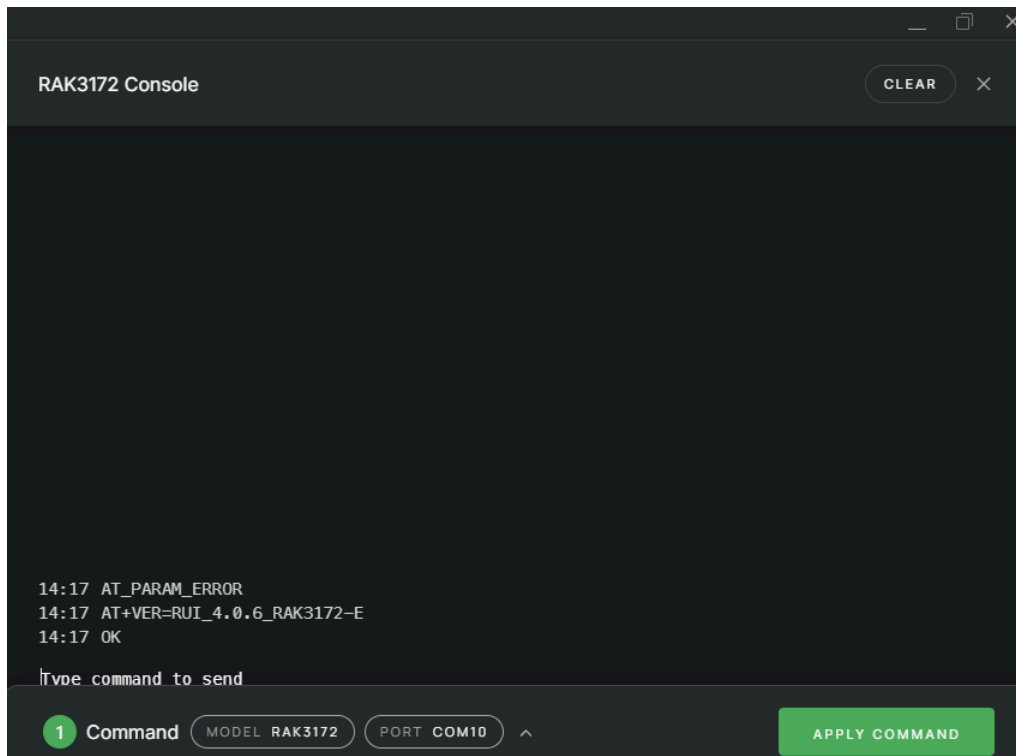
Identificación automática de dispositivos conectados.



Selección Manual de dispositivo (Usar los mismos parámetros).



Información del dispositivo.

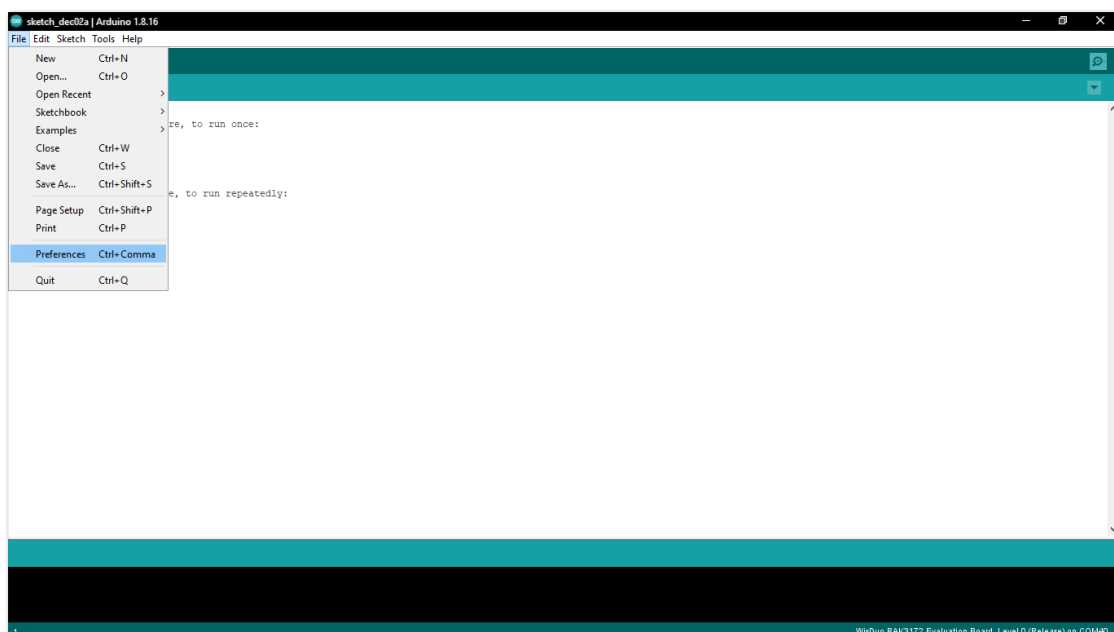


Prueba de comandos por consola.

10. Paquete de soporte de la placa RAK3272S RUI3 en Arduino IDE

También se tiene como opción la conexión con el dispositivo RAK usando el IDE de Arduino, similar al ESP32.

1. Open Arduino IDE and go to File > Preferences.

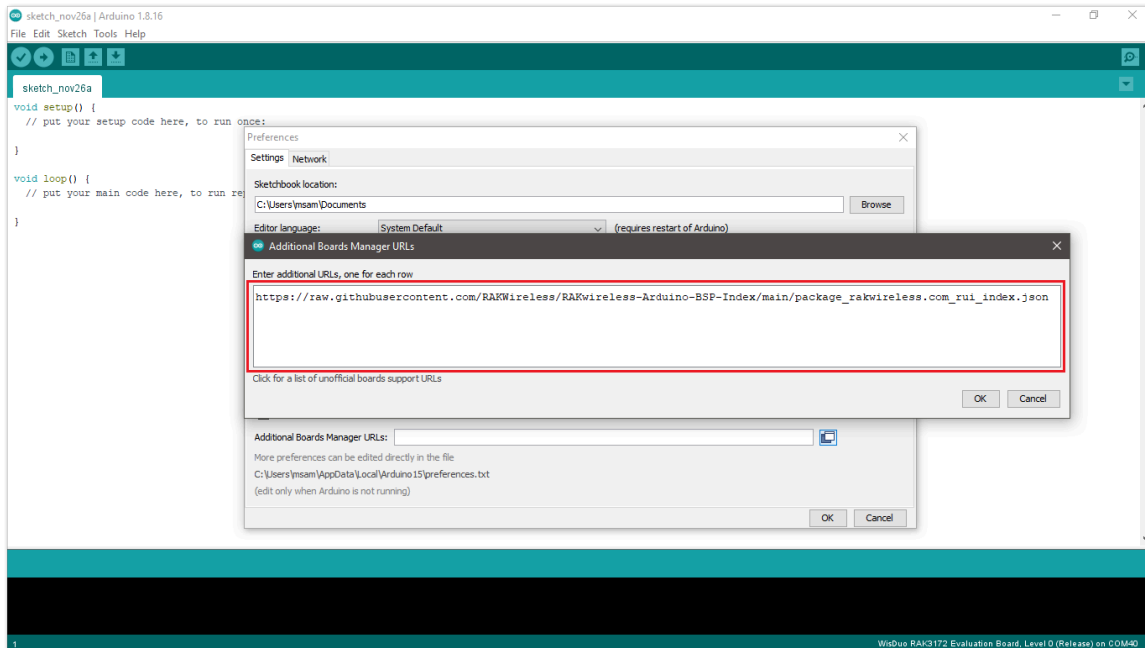


2. To add the RAK3272S to your Arduino Boards list, edit the Additional Board Manager URLs and click the icon, as shown in Figure 5.

3. Copy the URL

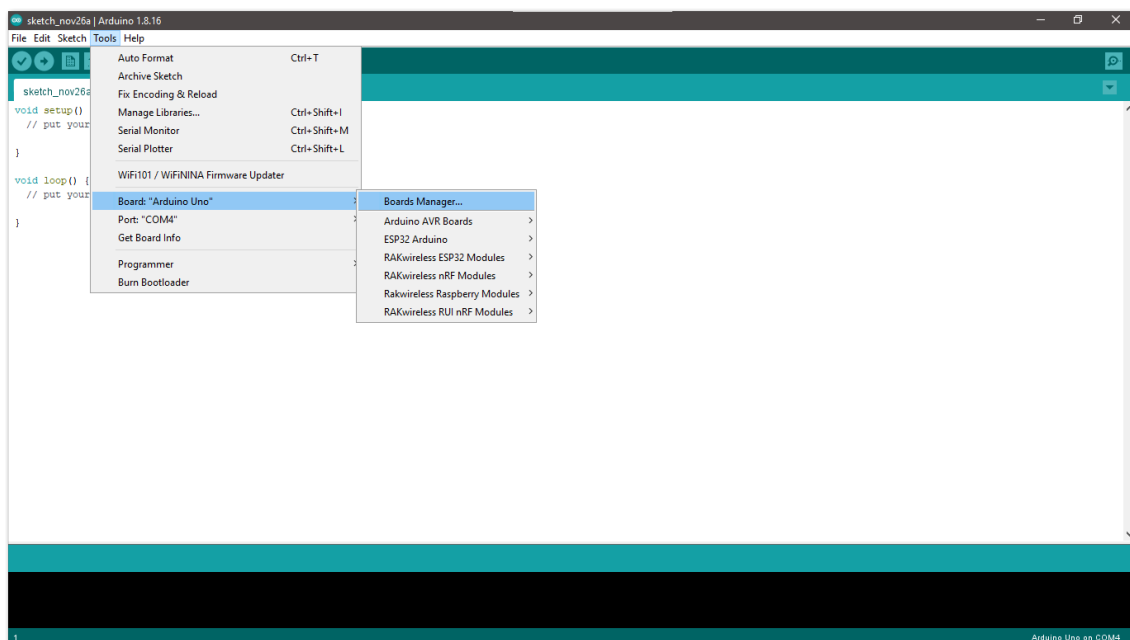
https://raw.githubusercontent.com/RAKWireless/RAKwireless-Arduino-BSP-Index/main/package_rakwireless_com_rui_index.json

and paste it on the field. If there are other URLs already there, just add them on the next line. After adding the URL, click OK.

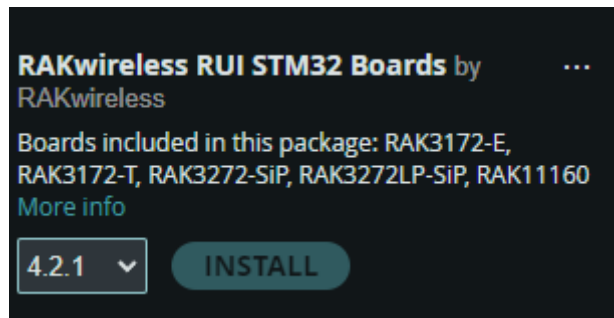


4. Restart the Arduino IDE.

5. Open the Boards Manager from Tools Menu.

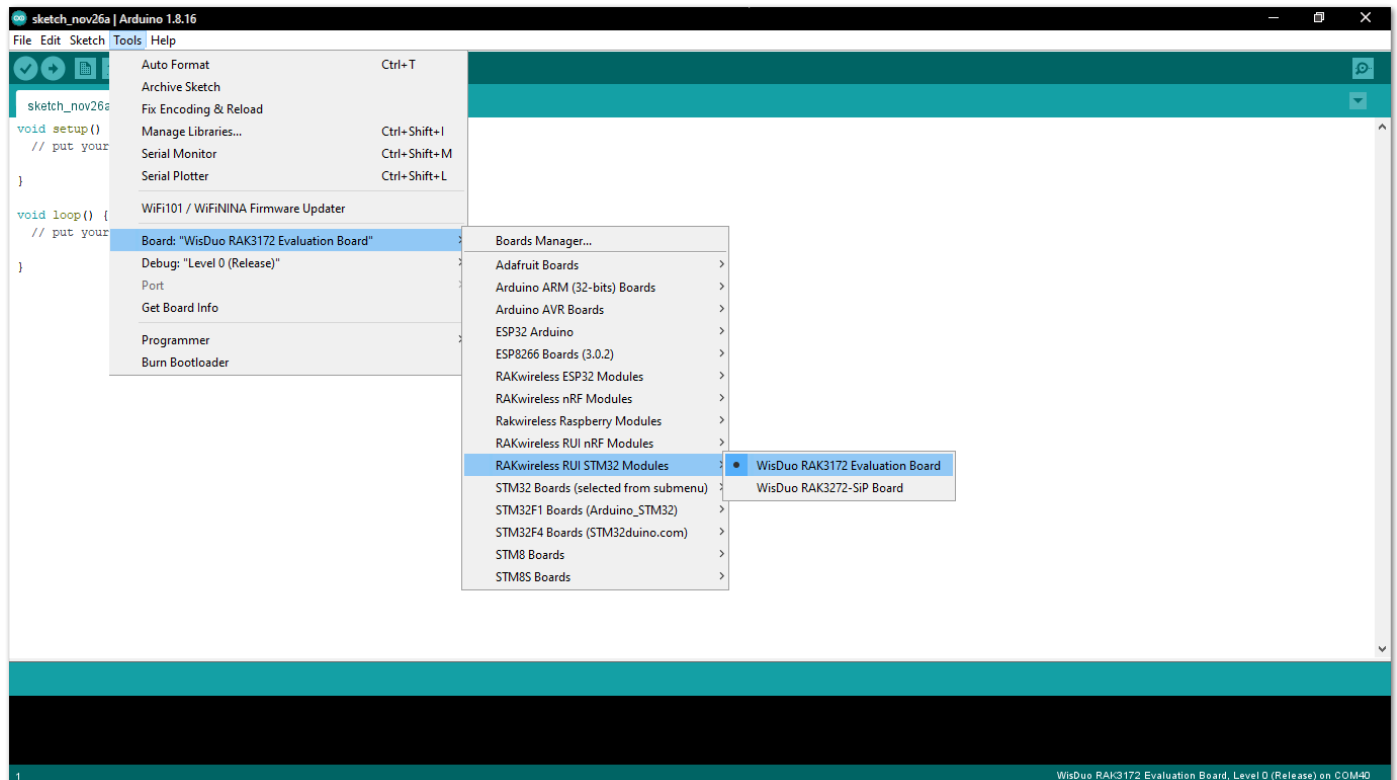


6. Type RAK in the search bar, as shown in Figure 8. This will display the available RAKwireless module boards that can be added to your Arduino board list. Select and install the latest version of the RAKwireless RUI STM32 Boards.

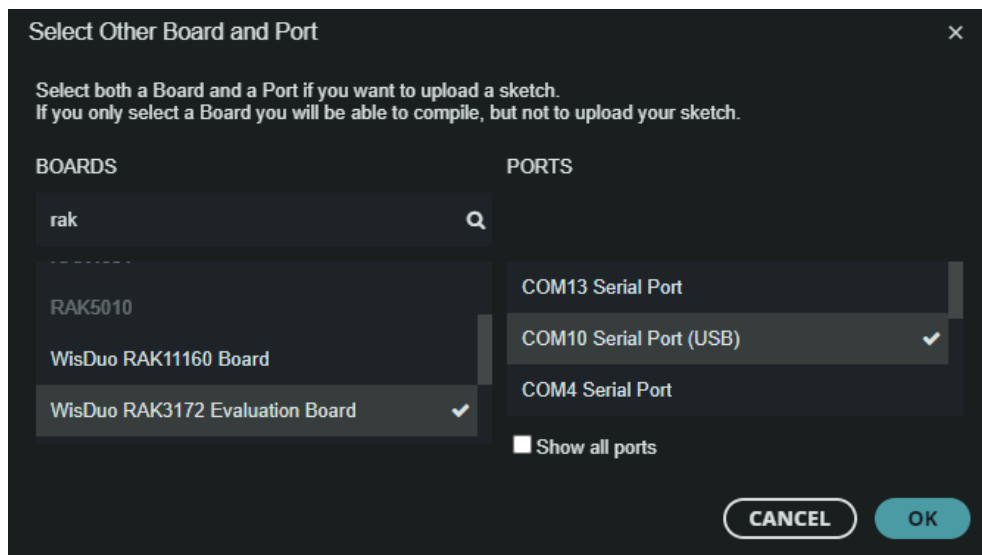


```
Output
Downloading packages
rak_rui:arm-none-eabi-gcc@9-2019q4
rak_rui:uploader_ymodem@1.0.1
rak_rui:buildtime@v0.0.0
rak_rui:stm32@4.2.1
Installing rak_rui:arm-none-eabi-gcc@9-2019q4
Configuring tool.
rak_rui:arm-none-eabi-gcc@9-2019q4 installed
Installing rak_rui:uploader_ymodem@1.0.1
Configuring tool.
rak_rui:uploader_ymodem@1.0.1 installed
Installing rak_rui:buildtime@v0.0.0
Configuring tool.
rak_rui:buildtime@v0.0.0 installed
Installing platform rak_rui:stm32@4.2.1
Configuring platform.
Platform rak_rui:stm32@4.2.1 installed
Successfully installed platform RAKwireless RUI STM32 Boards:4.2.1
```

- Once the BSP is installed, select Tools > Boards Manager > RAKWireless RUI STM32 Modules > WisDuo RAK3172 Evaluation Board. The RAK3272S Breakout Board uses RAK3172 WisDuo module.

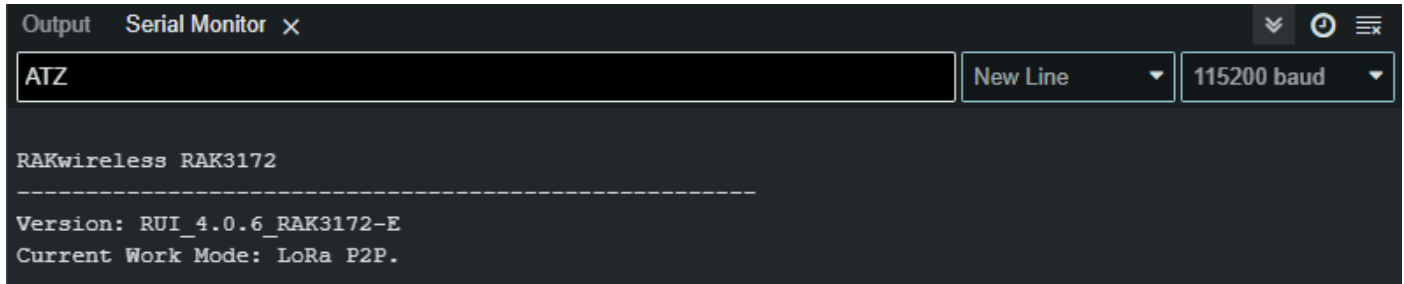


8. Select the board on the serial (COM) port where it is connected and configure the device as RAK3172.



9. Open the serial monitor and try sending an AT command (e.g. ATZ)

Nota: Por defecto el baudrate de estos dispositivos es de 115200.



```
Output  Serial Monitor x
ATZ
New Line 115200 baud

RAKwireless RAK3172
-----
Version: RUI_4.0.6_RAK3172-E
Current Work Mode: LoRa P2P.
```

11. Comandos AT (RUI v3)

Nota: Toda la información extendida acerca de los comandos AT se encuentra en el [repositorio de RAKWireless](#).

La RAK3272S Breakout Board está desarrollada en torno al chip STM32WLE5CC y está diseñada para simplificar la comunicación LoRaWAN y LoRa punto a punto (P2P).

Para integrar la tecnología LoRa en sus proyectos, la RAK3272S proporciona una interfaz de comunicación UART fácil de usar, que le permite enviar comandos AT. Estos comandos le permiten configurar parámetros para la comunicación LoRa P2P y LoRaWAN. Además, se puede utilizar cualquier microcontrolador con una interfaz UART para controlar la placa RAK3272S.

La interfaz de comunicación serie UART está disponible en UART2 (también identificado como el puerto LPUART1), accesible a través del Pin 7 (TX2) y el Pin 8 (RX2). La configuración por defecto de la comunicación UART2 es 115200 baudios, 8 bits de datos, sin paridad y 1 bit de parada (8-N-1). Las actualizaciones de firmware también se pueden realizar a través de este puerto. Para más detalles sobre la distribución de pines y un esquema de aplicación de referencia, consulte la [hoja de datos](#) de la placa de interconexión RAK3272S.

11.1. RUI3 Formato de comandos AT

Los comandos AT tienen el formato estándar AT+XXX, donde XXX denota el comando. Hay cuatro comportamientos de comando disponibles:

- **AT+XXX?** : proporciona una breve descripción del comando dado, por ejemplo, AT+DEVEUI?
- **AT+XXX** : se utiliza para ejecutar un comando, como AT+JOIN.
- **AT+XXX=?** : se utiliza para obtener el valor de un comando dado, por ejemplo, AT+CFS=?
- **AT+XXX=<valor>** : se utiliza para proporcionar un valor a un comando, por ejemplo, AT+CFM=1.

El formato de salida es el siguiente:

```
AT+XXX=<value><CR><LF>
<CR><LF><Status><CR><LF>
```

<CR> significa «carriage return» y <LF> significa «line feed».

La salida <value><CR><LF> se devuelve siempre que se ejecutan los comandos «help AT+XXX?» u «get AT+XXX=?».

Los posibles códigos de estado son:

- **OK:** el comando se ejecuta correctamente sin errores.
- **AT_ERROR:** error genérico.
- **AT_PARAM_ERROR:** un parámetro del comando es incorrecto.
- **AT_BUSY_ERROR:** la red LoRa está ocupada, por lo que el comando no se ha completado.
- **AT_TEST_PARAM_OVERFLOW:** el parámetro es demasiado largo.
- **AT_NO_CLASSB_ENABLE:** el nodo final aún no se ha conectado en clase B.
- **AT_NO_NETWORK_JOINED:** la red LoRa aún no se ha conectado.
- **AT_RX_ERROR:** detección de error durante la recepción del comando.

```
14:17 AT_PARAM_ERROR
14:17 AT+VER=RUI_4.0.6_RAK3172-E
14:17 OK
```

Ejemplo de ejecución de comandos.

Comando	Valor de Entrada	Valor de Respuesta	Descripción
AT	-	OK	Comando que permite verificar que la comunicación con el dispositivo es funcional.
ATZ	-	OK	Reinicia el módulo y brinda la información del dispositivo.
ATR	-	OK	Restaura todos los parámetros a los valores iniciales por defecto.
AT+SN=?	-	<1-18char>	Numero serial del dispositivo.
AT+VER=?	-	AT+VER=<ver>	Versión del firmware del dispositivo.
AT+SLEEP	-	OK	Activa el modo sleep.
AT+SLEEP=<val>	(Int) Tiempo en ms	OK	Activa el modo sleep durante el tiempo especificado en ms.
AT+BAUD=?		AT+BAUD=<bauds>	Obtiene la velocidad en baudios del puerto serie.
AT+BAUD=<val>	(Int) Velocidad en baudios.	OK	Configurar la velocidad en baudios del puerto serie.

Comando	Valor de Entrada	Valor de Respuesta	Descripción
AT+NWM=<0,1>	1 Bit	OK	Comando que configura el modo de red empleado por el dispositivo. 0: LoRa P2P. 1: LoRaWAN.
AT+P2P=<f>:<sf>:<bw>:<cr>:<pl>:<p>	Varios valores.	OK	Comando de configuración del modo P2P, donde: f: Frecuencia de enlace. Sf: Spreading Factor. bw: Ancho de banda en kHz. cr: Tasa de codificación. pl: Longitud del preámbulo. p: Potencia en dBm.
AT+PRECV=<t>	Tiempo ms	OK	Comando para establecer la duración de la recepción en ms. Donde hay casos especiales: 0: El dispositivo funcionará únicamente como TX. 65535: Funcionará como RX hasta que reciba un mensaje, y luego cambia a TX (0). 65534: Funciona únicamente como RX, hasta volver a configurarlo.
AT+PSEND=<payload>	Mensaje Hexadecimal	OK	Comando para enviar un mensaje <payload> en formato hexadecimal compuesto por hasta 256 caracteres hexadecimales.

Comandos RUI3 empleados en comunicación LoRa P2P.

12. Sensor de Gas MQ2

La familia de sensores MQ son sensores especializados en detección de componentes químicos en aire, es decir, mediciones de gas en general. El Módulo Sensor MQ-2 permite medir concentraciones de gas GLP y GNV en el aire. El MQ-2 es sensible a GLP, i-butano, propano, metano, alcohol, hidrógeno (H₂) y humo.

El módulo posee una salida analógica que proviene del divisor de voltaje que forma el sensor y una resistencia de carga. También tiene una salida digital que se calibra con un potenciómetro, esta salida tiene un LED indicador. La resistencia interna del sensor cambia de acuerdo a la concentración del gas en el aire.

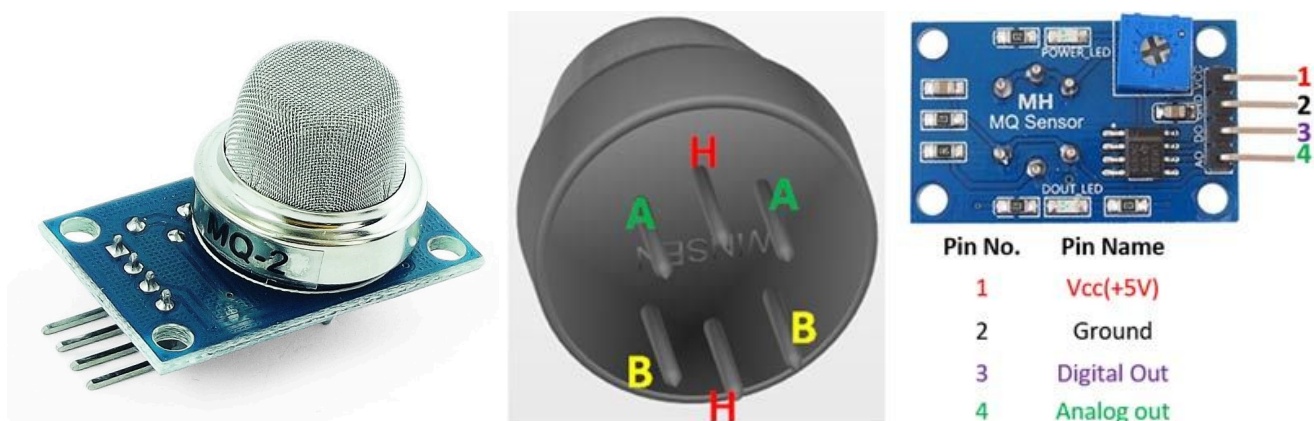


Fig. A. Fotografía referencial y Pinout de sensor de gas MQ2.

A continuación se deja un enlace a su hoja de datos. [Datasheet](#)

En esta hoja de datos, podemos encontrar su gráfica de sensibilidad para diferentes tipos de elementos en forma de gas para partículas por millón. Se basa en una relación de división entre la resistencia que emplea el sensor como referencia y la resistencia medible, la cual varía de acuerdo a la presencia de los gases. Para fines prácticos, se recomienda el uso de una librería para la familia de sensores MQ, la cuál se muestra en la Fig. X, dado que para el proceso de medición en ppm es necesario hacer una relación lineal acorde a la gráfica. Con el uso de la librería, la función **MQX.ReadSensor()** brinda la medición en ppm.

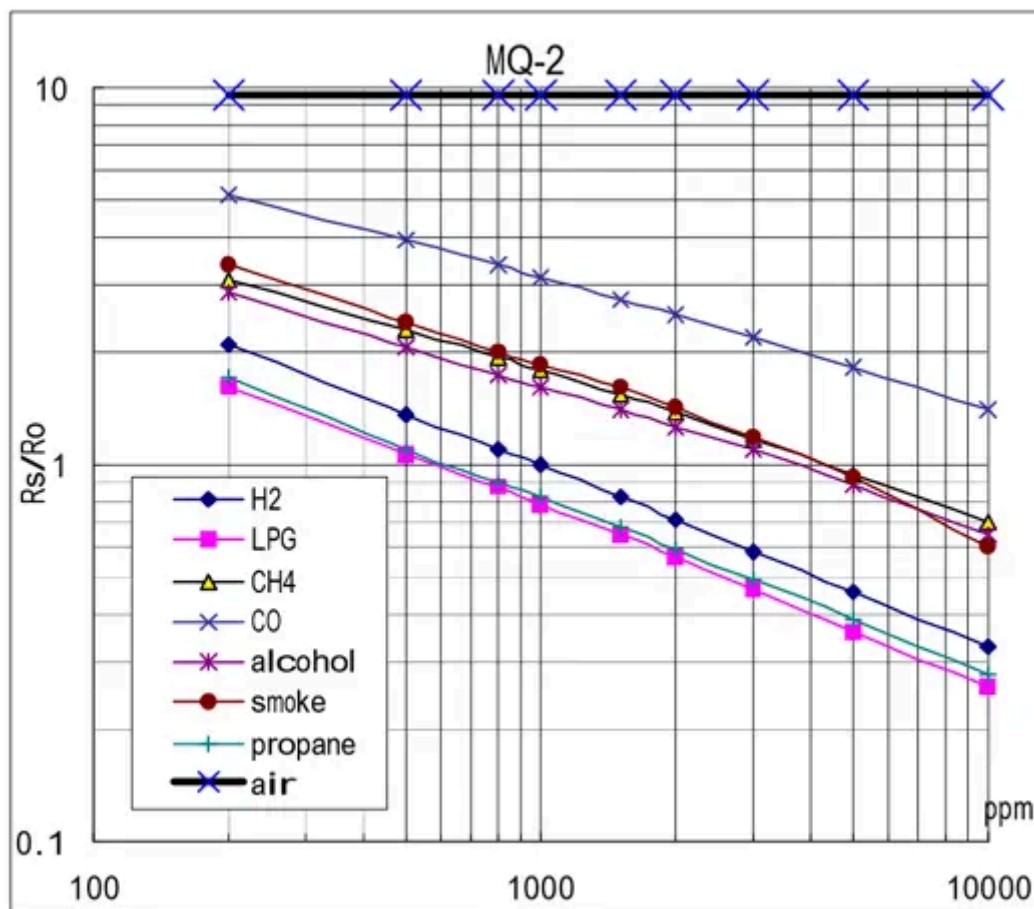


Fig. B. Tabla de sensibilidad vs. concentración en ppm de los diferentes tipos de gases.

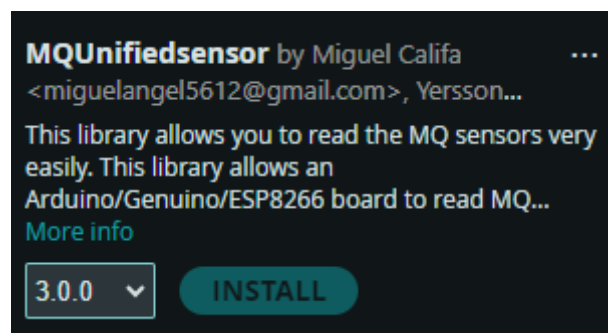


Fig. C. Librería para sensores MQ en Arduino.

13. Código de Ejemplo Arduino + RAK

Este código sirve de ejemplo para poder enviar comandos AT a través de los puertos Serial1 (TX1 y RX1) de Arduino, los cuales deben ser conectados al RAK3272 a través de un divisor de voltaje (similar a lo visto en ESP32).

```
int num = 20;
String hexNum;
void setup() {
    // initialize both serial ports:
    Serial.begin(115200);
    Serial1.begin(115200);
    delay(1000);
    sendRAK("at+ver=?");
    delay(500);
    sendRAK("at+ver=?");
    delay(500);
    sendRAK("at+NWM=0");
    delay(500);
    sendRAK("at+P2P=923000000:7:125:0:10:14");
    delay(500);
    sendRAK("at+PRECV=0");
    delay(500);
}

void loop() {
    num = num + 1;
    hexNum = StringToHex(String(num));
    sendRAK("at+PSEND=" + String(hexNum));

    // Wait for a second before sending the next message
    delay(5000);
}

void sendRAK(String message) {
    // Define the string to be sent
    Serial.print("AT COMMAND SENT: ");
    Serial.println(message);

    // Send the string through Serial1
    Serial1.println(message);

    // Optionally, print the message to the Serial Monitor for debugging
    Serial.println("Sent: " + message);
}
```

```
// Wait for a response
while (Serial1.available() == 0) {
    // Do nothing and wait for data to become available
}

// Read the response from Serial1
String response = "";
while (Serial1.available() > 0) {
    response += (char)Serial1.read();
}

// Print the response to the Serial Monitor
Serial.println("Received: " + response);
}

String stringToHex(String str) {
    String hexString = "";
    for (int i = 0; i < str.length(); i++) {
        char hex[3]; // Two characters for the hex code plus the null terminator
        sprintf(hex, "%02X", str[i]);
        hexString += hex;
    }
    return hexString;
}
```

14. Código Ejemplo de encendido o apagado de LED en RAK como Receptor (Deprecated)

Este código funciona de manera que recibe el mensaje en HEX, lo traduce (con código de ejemplo de RAK3272S) y si encuentra el mensaje "UNO" (que es el programado de envío para transmisor) enciende el LED conectado a PA7.

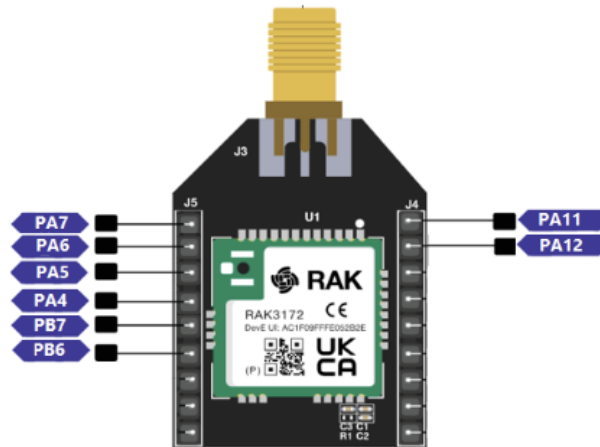


Figure 23: Available Digital I/O pins in RAK3272S

```
/* *****  
 * This example is from https://github.com/Kongduino/RUI3_LoRa_P2P_PING_PONG  
 * This example needs two devices, a device will send LoRa P2P package after  
receiving LoRa P2P package.  
 ***** */  
  
long startTime;  
bool rx_done = false;  
double myFreq = 923000000;  
uint16_t sf = 7, bw = 125, cr = 0, preamble = 10, txPower = 14;  
  
void hexDump(uint8_t * buf, uint16_t len)  
{  
    char *str = (char *)buf;  
    char alphabet[17] = "0123456789abcdef";  
    Serial.print(F("          +-----+  
+-----+\r\n"));  
    Serial.print(F("      |.0 .1 .2 .3 .4 .5 .6 .7 .8 .9 .a .b .c .d .e .f | |  
ASCII      |\r\n"));  
    for (uint16_t i = 0; i < len; i += 16) {  
        if (i % 128 == 0)  
            Serial.print(F("          +-----+  
+-----+\r\n"));  
        char s[] = " | |  
|\r\n";  
        uint8_t ix = 1, iy = 52;  
        for (uint8_t j = 0; j < 16; j++) {  
            if (i + j < len) {  
                uint8_t c = buf[i + j];
```

```
        s[ix++] = alphabet[(c >> 4) & 0x0F];
        s[ix++] = alphabet[c & 0x0F];
        ix++;
        if (c > 31 && c < 128)
            s[iy++] = c;
        else
            s[iy++] = '.';
    }
}

uint8_t index = i / 16;
if (i < 256)
    Serial.write(' ');
Serial.print(index, HEX);
Serial.write('.');
Serial.print(s);
}

    Serial.print(F("      +-----+
+-----+\\r\\n"));

    Serial.println(str);

// PRENDE O APAGA PIN PA7 SI ENCUENTRA MENSAJE "UNO" EN STRING
if(strstr(str, "UNO")){
    digitalWrite(PA7, HIGH);
    delay(1000);
}
else {
    digitalWrite(PA7, LOW);
    delay(1000);
}

}

/*
typedef struct rui_lora_p2p_rev {
// Pointer to the received data stream
uint8_t *Buffer;
// Size of the received data stream
uint8_t BufferSize;
// Rssi of the received packet
int16_t Rssi;
// Snr of the received packet
```

```
int8_t Snr;
} rui_lora_p2p_rcv_t;
*/
void rcv_cb(rui_lora_p2p_rcv_t data)
{
    rx_done = true;
    if (data.BufferSize == 0) {
        Serial.println("Empty buffer.");
        return;
    }
    char buff[92];
    sprintf(buff, "Incoming message, length: %d, RSSI: %d, SNR: %d",
        data.BufferSize, data.Rssi, data.Snr);
    Serial.println(buff);
    hexDump(data.Buffer, data.BufferSize);
}

void send_cb(void)
{
    Serial.printf("P2P set Rx mode %s\r\n",
        api.lora.prcv(65534) ? "Success" : "Fail");
}

void setup()
{
    pinMode(PA7, OUTPUT);

    Serial.begin(115200);
    Serial.println("RAKwireless LoRaWan P2P Example");
    Serial.println("-----");
    delay(2000);
    startTime = millis();

    if(api.lora.nwm.get() != 0)
    {
        Serial.printf("Set Node device work mode %s\r\n",
            api.lora.nwm.set() ? "Success" : "Fail");
        api.system.reboot();
    }

    Serial.println("P2P Start");
    Serial.printf("Hardware ID: %s\r\n", api.system.chipId.get().c_str());
    Serial.printf("Model ID: %s\r\n", api.system.modelId.get().c_str());
}
```

```
Serial.printf("RUI API Version: %s\r\n",
    api.system.apiVersion.get().c_str());
Serial.printf("Firmware Version: %s\r\n",
    api.system.firmwareVersion.get().c_str());
Serial.printf("AT Command Version: %s\r\n",
    api.system.cliVersion.get().c_str());
Serial.printf("Set P2P mode frequency %3.3f: %s\r\n", (myFreq / 1e6),
    api.lora.pfreq.set(myFreq) ? "Success" : "Fail");
Serial.printf("Set P2P mode spreading factor %d: %s\r\n", sf,
    api.lora.psf.set(sf) ? "Success" : "Fail");
Serial.printf("Set P2P mode bandwidth %d: %s\r\n", bw,
    api.lora.pbw.set(bw) ? "Success" : "Fail");
Serial.printf("Set P2P mode code rate 4/%d: %s\r\n", (cr + 5),
    api.lora.pcr.set(cr) ? "Success" : "Fail");
Serial.printf("Set P2P mode preamble length %d: %s\r\n", preamble,
    api.lora.ppl.set(preamble) ? "Success" : "Fail");
Serial.printf("Set P2P mode tx power %d: %s\r\n", txPower,
    api.lora.ptp.set(txPower) ? "Success" : "Fail");
api.lora.registerPRecvCallback(recv_cb);
api.lora.registerPSendCallback(send_cb);
Serial.printf("P2P set Rx mode %s\r\n",
    api.lora.precv(3000) ? "Success" : "Fail");
// let's kick-start things by waiting 3 seconds.
}

void loop()
{
    uint8_t payload[] = "payload";
    bool send_result = false;
    if (rx_done) {
        rx_done = false;
        while (!send_result) {
            send_result = api.lora.psend(sizeof(payload), payload);
            Serial.printf("P2P send %s\r\n", send_result ? "Success" : "Fail");
            if (!send_result) {
                Serial.printf("P2P finish Rx mode %s\r\n", api.lora.precv(0) ?
"Success" : "Fail");
                delay(1000);
            }
        }
    }
    delay(500);
}
```

15. Recepción de mensajes RAK con Python

Este anexo tiene la información necesaria para establecer comunicación UART entre una computadora y un microcontrolador (p. ej. ESP32, Arduino, RAK3272S) utilizando Python. Se emplea la biblioteca “**pyserial**” para enviar y recibir mensajes a través del puerto serie.

15.1. Requisitos

- Python 3.x instalado en el sistema
- Biblioteca **pyserial** (pip install **pyserial**)
- Tener el dispositivo conectado mediante UART al computador.
- IDE

15.2. Instalación de Pyserial

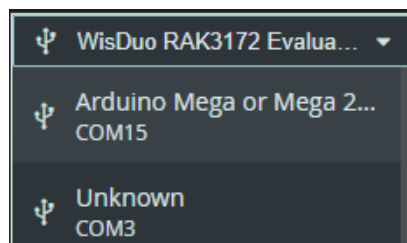
Para instalar pyserial, ejecutar en terminal:

```
pip install pyserial
```

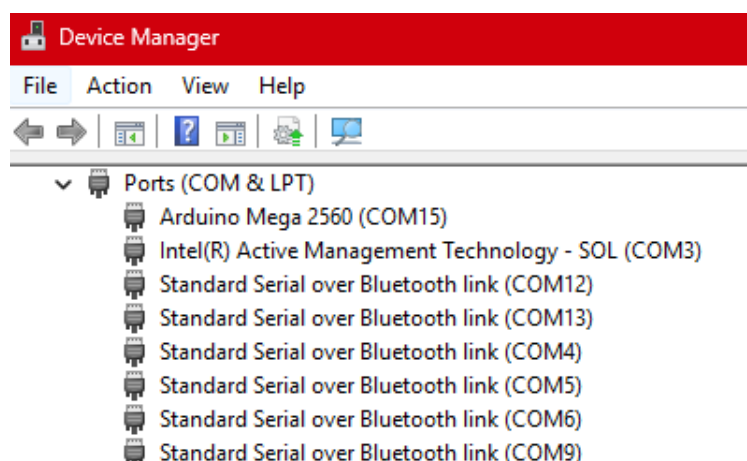
```
C:\Users\gabri>pip install pyserial
Collecting pyserial
  Downloading pyserial-3.5-py2.py3-none-any.whl.metadata (1.6 kB)
  Downloading pyserial-3.5-py2.py3-none-any.whl (90 kB)
Installing collected packages: pyserial
Successfully installed pyserial-3.5
```

15.3. Detección del puerto serial

Puede comprobarlo mediante su IDE de Arduino, al conectar su dispositivo mediante UART.



En Windows, alternativamente se puede verificar mediante el administrador de dispositivos.



Una vez identificado el puerto, se procede a la programación en python.

15.4. Código Básico para Enviar y Recibir Mensajes UART

A continuación se muestra un código de ejemplo de comunicación serial, que recibe mensajes de un Arduino MEGA conectado en el puerto COM15 a 115200.

```
import serial
import time

# Configuración del puerto serie
puerto = serial.Serial(
    port='COM15',          # Cambiar por el puerto correspondiente
    baudrate=115200,       # Velocidad de baudios
    timeout=1              # Tiempo de espera para lectura
)

time.sleep(2) # Espera para establecer conexión con el microcontrolador

# Enviar un mensaje
mensaje = "Hola desde Python\n"
puerto.write(mensaje.encode('utf-8')) # Codifica a bytes

# Leer respuesta del microcontrolador
while True:
    if puerto.in_waiting > 0:
        respuesta = puerto.readline().decode('utf-8').strip()
        print("Recibido:", respuesta)
        break
puerto.close()
```

```
Recibido: AT COMMAND SENT: at+ver=?
```